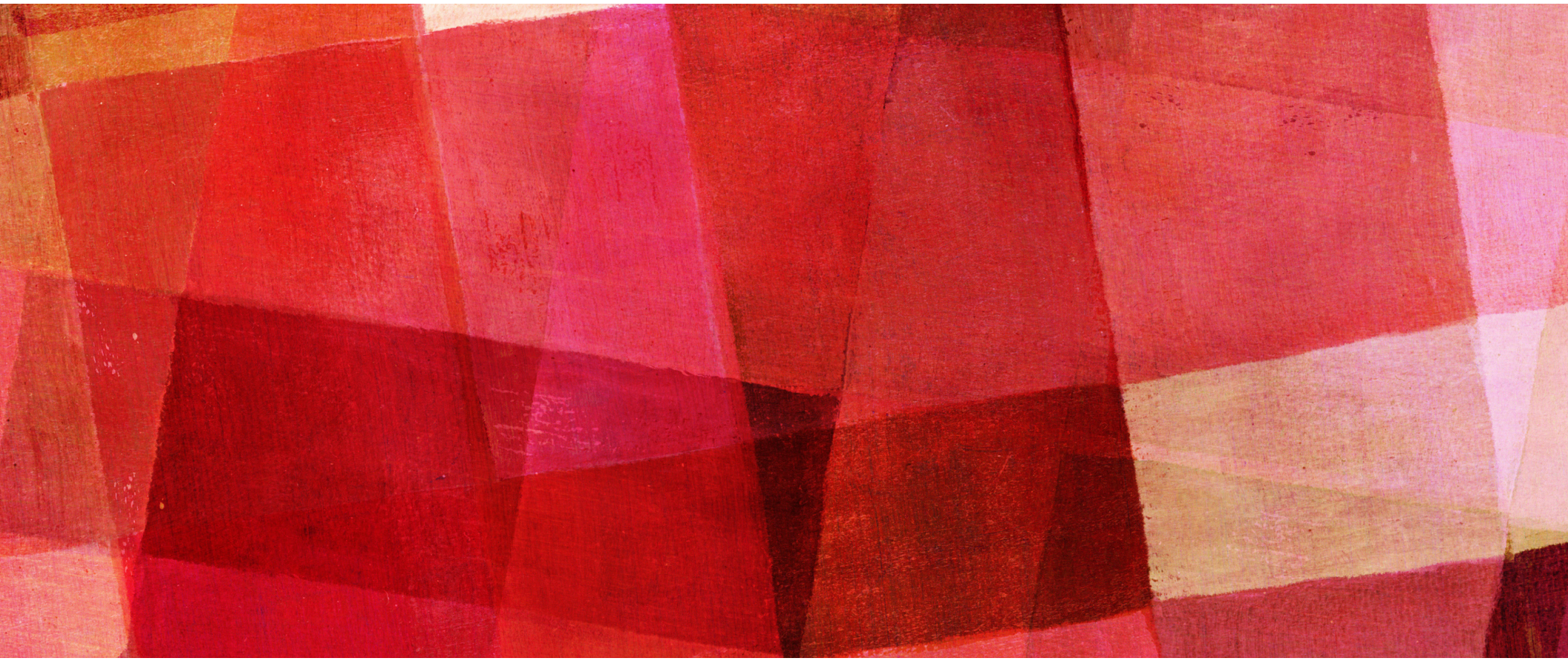


COM S 228 RECITATION 2

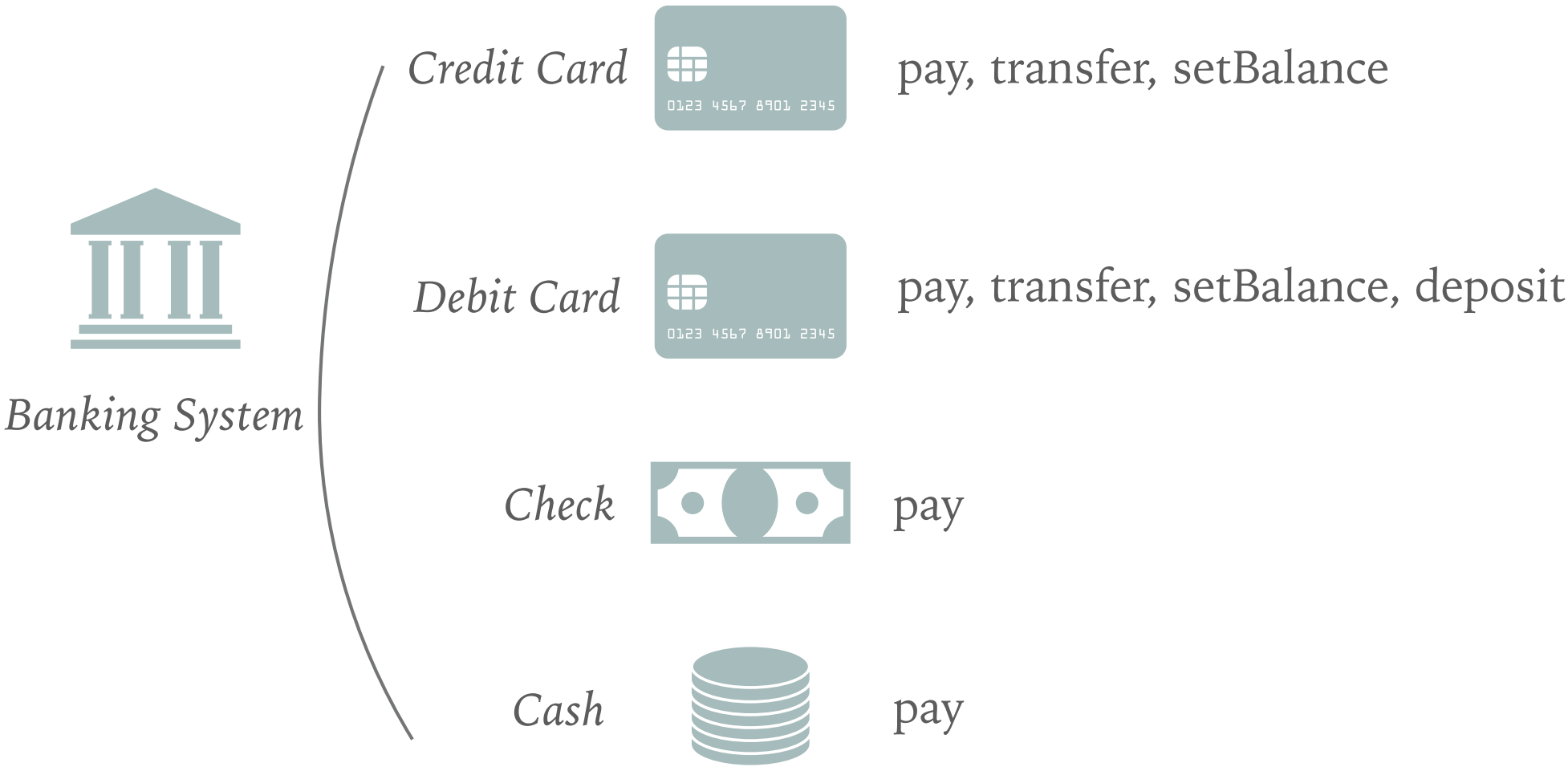
Inheritance, instanceof and getClass()

INHERITANCE

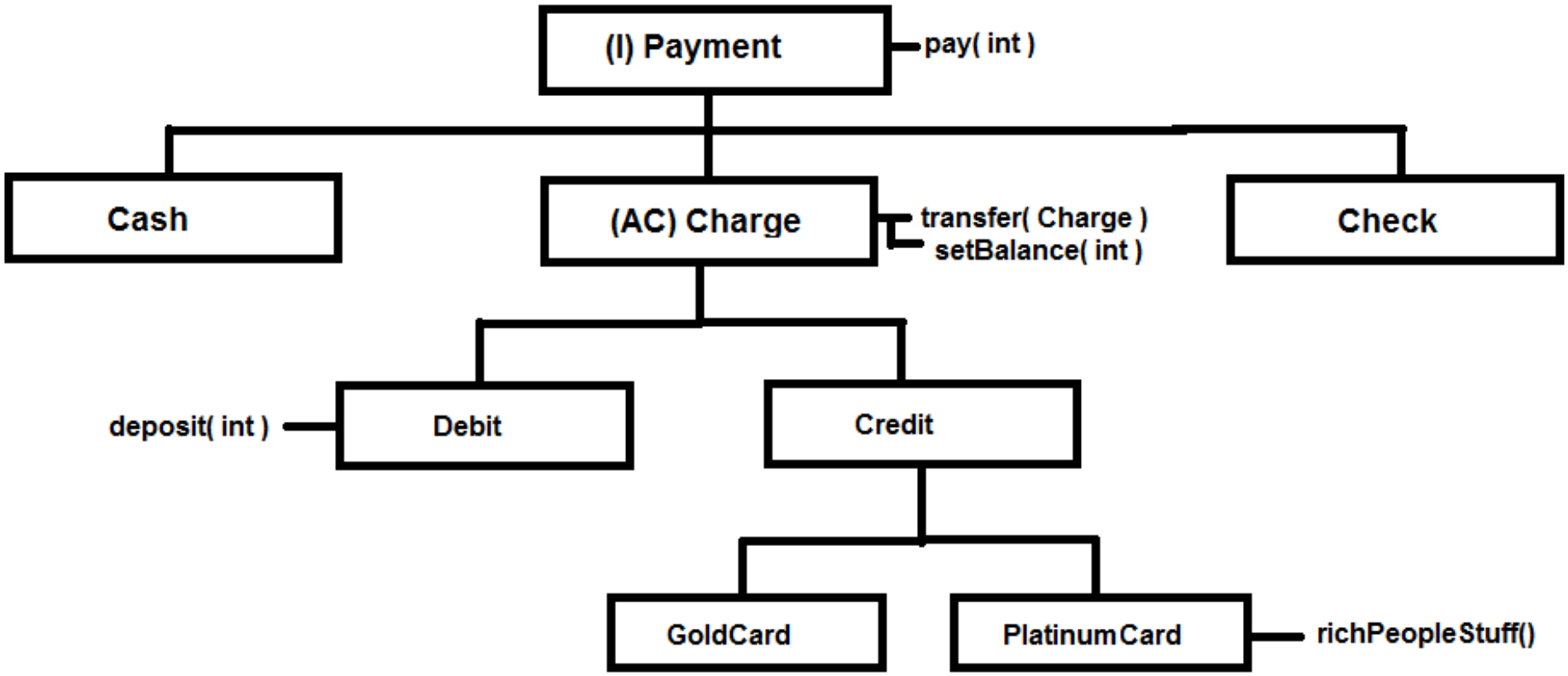
Inheritance can be defined as the process where one class acquires the properties (methods and fields) of another. With the use of inheritance the information is made manageable in a hierarchical order.

The class which inherits the properties of other is known as subclass (derived class, child class) and the class whose properties are inherited is known as superclass (base class, parent class).

Suppose you need to develop a banking system:



INHERITANCE – AN EXAMPLE



INHERITANCE – ABSTRACT CLASS & INTERFACE

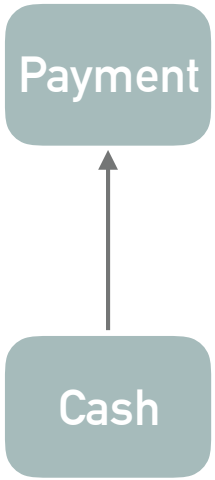
Interfaces are often used to describe the peripheral abilities of a class, not its central identity.

An abstract class defines the core identity of its descendants.

INHERITANCE – ABSTRACT CLASS & INTERFACE

Features	Interface	Abstract Class
multiple inheritance	A class may implement **several** interfaces.	A class may extend **only one** abstract class.
default implementation	An interface cannot provide any code at all, much less default code.	An abstract class can provide complete code, default code and/or just stubs that have to be overridden.
constants	Static final constants only	Both instance and static constants are possible
speed	Slow	Fast

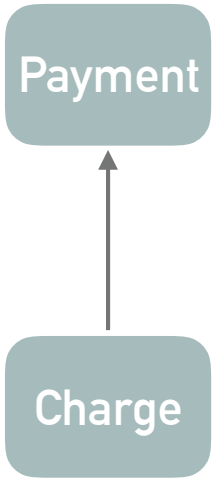
INHERITANCE – AN EXAMPLE




```
// Payment.java

public interface Payment
{
     public void pay(int amount);
}
```

```
// Cash.java

public class Cash implements Payment
{
     private int bill;

     public Cash() {
          this( 50 );
     }

     public Cash( int b ) {
          bill = b;
     }

     public void pay( int amount ) {
          if( amount > 50 )
            System.out.println( "you get " + ( amount - 50 ) + " in change" );
          else
            System.out.println( "you still owe " + ( 50 - amount ) + "!" );
     }
}
```

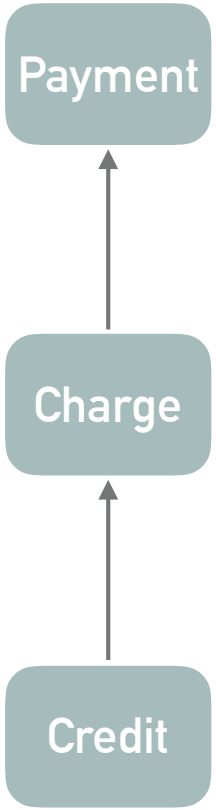
INHERITANCE – AN EXAMPLE




```
// Payment.java

public interface Payment
{
     public void pay(int amount);
}
```

```
// Charge.java

public abstract class Charge implements Payment
{
     protected int balance;
     protected boolean active;

     public Charge( int b ) { balance = b; active = true; }

     // this method transfers the balance from the current card to the new card.
     public void transfer( Charge newCard )
     {
          newCard.setBalance( this.balance );
          this.balance = 0;
          this.active = false;
     }

     // sets the balance on the card
     private void setBalance( int newBalance ) {balance = newBalance;}

     protected boolean isActive() {return active;}
}
```

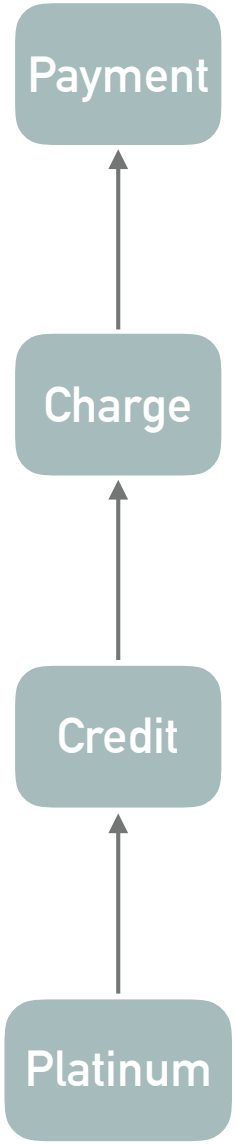
INHERITANCE – AN EXAMPLE




```
// Charge.java

public abstract class Charge implements Payment
{
     public void pay(int amount);
}
```

```
// Credit.java

public class Credit extends Charge
{
     private int limit;

     public Credit() { super( 0 ); limit = 1000;}
     public Credit( int l ) { super( 0 ); limit = l; }

     @Override
     public void pay( int amount ) {
          // can I charge?
          if( !this.isActive() || ( amount + balance ) > limit )
          {
               System.out.println( "Declined." );
               return;
          }

          balance += amount;
          System.out.println( "Approved!" );
     }
}
```

INHERITANCE – AN EXAMPLE




```java
// Credit.java

public class Credit extends Charge
{
     // ……
}
```

```java
// PlatinumCard.java

public class PlatinumCard extends Credit
{
     public PlatinumCard() { super( Integer.MAX_VALUE );}
     public PlatinumCard( int limit ){ super( limit ); }

     @Override
     public void pay( int amount )
     {
          super.pay( amount );
          // rich people get tons of benefits for no reason
          balance -= ( amount * 0.5 );
     }

     public void richPeopleStuff()
     {
          System.out.println( "Free yacht with purchase of sandwich." );
     }
}
```

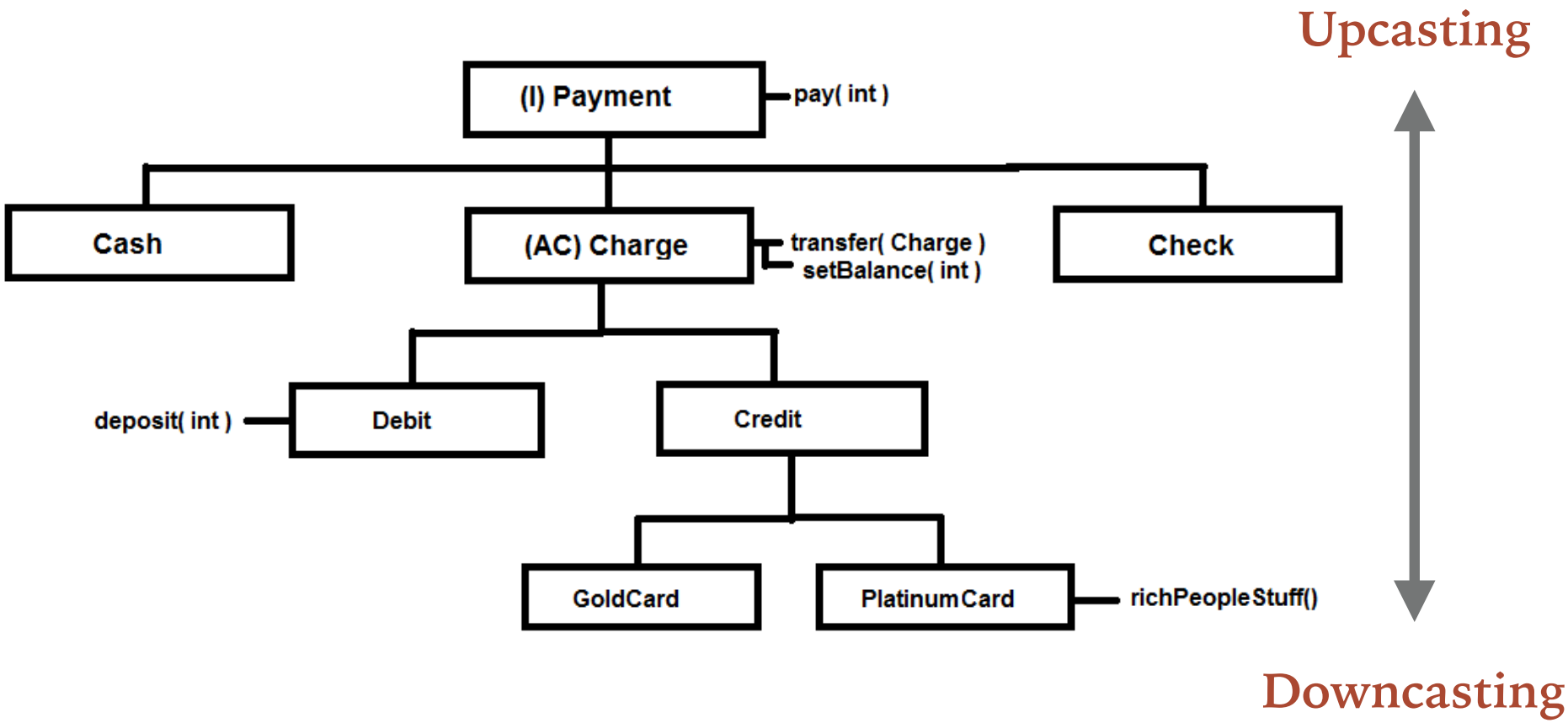
INHERITANCE – CAST DOWN/UP/SIDE

1. you are not actually changing the object itself, you are just labeling it differently

   For example, if you create a PlatinumCard and upcast it to CreditCard, then the object doesn't stop from being a PlatinumCard. It's still a PlatinumCard, but it's just treated as any other CreditCard and it's PlatinumCard properties (richPeopleStuff) are hidden until it's downcasted to a PlatinumCard again.

```
PlatinumCard americanexpress = new PlatinumCard();

// or

CreditCard americanexpress = new PlatinumCard();
```

INHERITANCE – CAST DOWN/UP/SIDE



INHERITANCE – STATIC/DYNAMIC BINDING

Dynamic(Late) Binding

```
class Human{
   public void walk()
   {
       System.out.println("Human walks");
   }
}

class Boy extends Human{
   public void walk(){
       System.out.println("Boy walks");
   }

   public static void main( String args[]) {
       // Reference is of parent class
       Human h = new Boy();
       h.walk();
       h = new Human();
       h.walk();
   }
}
```

```
Output:
                Boy walks
                Human walks
```

INHERITANCE – STATIC/DYNAMIC BINDING

Static(Early) Binding

```
class Human{
   public static void walk()
   {
       System.out.println("Human walks");
   }
}

class Boy extends Human{
   public static void walk(){
       System.out.println("Boy walks");
   }

   public static void main( String args[]) {
       // Reference is of parent class
       Human h = new Boy();
       h.walk();
       h = new Human();
       h.walk();
   }
}
```

```
Output:
                Human walks
                Human walks
```

We have created one object of subclass and one object of superclass with the reference of the superclass.

Since the walk method of superclass is static, compiler knows that it will not be overridden in subclasses and hence compiler knows during compile time which walk method to call and hence no ambiguity.

INHERITANCE – INSTANCEOF & GETCLASS()

instanceof is a useful tool when you've got a collection of objects and you're not sure what they are.

That is to say, using **instanceof** can make it safe to downcast.

```
if (obj instanceof PlatinumCard)
{
    PlatinumCard card = (PlatinumCard)obj;
    card.richPeopleStuff()
}
```

instanceof tests whether the object reference on the left-hand side (LHS) is an instance of the type on the right-hand side (RHS) or some subtype.

getClass() == ... tests whether the types are identical.

INHERITANCE – INSTANCEOF & GETCLASS()

getClass() == ... tests whether the types are identical.

```
PlatinumCard obj = new PlatinumCard

if (obj.getClass() == CreditCard.class)
{
   System.out.println("obj is equal to CreditCard.")
}
else
{
   System.out.println("obj is not equal to CreditCard.");
}
```

Match a class only: use getClass() ==

Match a class and its subclass: use instanceof